

C++ 핵심 요약 정리본

기초 문법 정의 및 필수 소스코드 구조 요약

입출력 및 프로그램의 기본 구조

프로그램의 시작점, 헤더 파일 포함, 주석 작성법, 세미콜론의 사용, 그리고 표준 입출력(cout, cin)의 사용법을 포함한 기본 코드 틀이다.

```
#include <iostream> // 입출력 기능을 사용하기 위한 표준 라이브러리 헤더

// 프로그램의 실행이 시작되는 기본 함수이다
int main() {
    // 한 줄 주석은 이렇게 작성한다
    /* 여러 줄 주석은
       이 기호 사이에 작성한다
    */

    int input_value;

    // 콘솔 화면에 데이터를 출력한다 (문장 끝에는 세미콜론이 필수이다)
    std::cout << "숫자를 입력하세요: ";

    // 키보드로부터 사용자 입력을 받아 변수에 저장한다
    std::cin >> input_value;

    std::cout << "입력한 숫자: " << input_value << std::endl;

    return 0;
}
```

데이터 자료형 및 연산자

- **int** : 정수 값 (ex. 5, 10, 100)
- **double** : 소수점이 있는 실수 값 (ex. 1.25, 3.7, 3.141592)
- **bool** : 참 또는 거짓 값 (ex. true, false)
- **char** : 문자 하나 값 (ex. 'A', 'B')
- **std::string** : 문자열 값 (ex. "turtlebot3", "robot")
- **나머지 연산자 (%)** : 나눗셈 연산 후의 나머지 값을 구함 (ex. 10 % 3 결과는 1)

• 대입(=)과 비교(==) :

- `=` : 우측의 값을 변수에 넣는 연산 (ex. `speed = 0;`)
- `==` : 두 값이 같은지 판단하는 연산 (ex. `if (speed == 0)`)

제어 흐름 (조건문, 반복문, 배열)

조건문 (if / else if / else) : 주어진 조건식의 참/거짓 여부에 따라 실행할 코드 블록을 분기하는 문법이다.

```
if (battery > 50) {  
    // 조건이 참일 때 실행  
} else if (mode == 2) {  
    // 다른 조건을 검사  
} else {  
    // 모든 조건이 거짓일 때 실행  
}
```

반복문 (for / while) : 특정 명령을 정해진 횟수만큼 또는 조건이 만족하는 동안 반복 수행하는 문법이다. while문은 내부에 조건을 변화시키는 코드(증감식)를 누락하면 무한 루프에 빠질 수 있다.

```
// for문: 정해진 횟수 반복  
for (int i = 0; i < 5; i++) {  
    std::cout << i;  
}  
  
// while문: 조건이 참인 동안 반복  
int count = 0;  
while (count < 5) {  
    std::cout << count;  
    count++; // 증가 코드 필수  
}
```

배열 (Array) : 같은 자료형의 데이터 여러 개를 연속된 공간에 저장하는 자료구조이다. 방 번호(인덱스)는 반드시 0부터 시작한다.

```
int values[3] = {10, 20, 30};  
values[0] = 15; // 첫 번째 요소에 접근
```

메모리 및 데이터 관리 (함수, 참조, 포인터)

함수 (Function) : 특정한 작업을 수행하는 코드를 하나의 이름으로 묶은 것이다. 반환 자료형이 있으면 반드시 return으로 값을 돌려주어야 하며, 반환값이 없을 때는 void를 쓴다. 호출 시에는 반드시 이름 뒤에 괄호()를 붙여야 한다.

```
int add(int a, int b) {
    return a + b;
}
```

참조 (&) : 기존 변수에 새로운 별명을 붙여 직접 접근하는 방식이다. 참조 매개변수를 쓰면 함수 내부에서 원본 변수의 값을 직접 변경할 수 있다.

```
void updateSpeed(int& speed) {
    speed = 100; // 원본 값이 바뀜
}
```

포인터 (*) : 데이터가 저장된 메모리의 주소값을 저장하는 특수 변수이다. 주소를 얻을 때는 &변수를 쓰고, 가리키는 주소의 실제 값에 접근할 때는 *포인터를 쓴다.

```
int speed = 10;
int* ptr = &speed; // 주소 저장
*ptr = 20;          // 주소에 있는 실제 값을 변경
```

nullptr : 포인터가 그 어떤 메모리 주소도 가리키고 있지 않음을 나타내는 안전한 빈 주소 값이다. 가리키는 대상이 있는지 검증한 뒤 사용해야 안전하다.

```
int* ptr = nullptr;
if (ptr != nullptr) {
    std::cout << *ptr;
}
```

객체지향 프로그래밍 (구조체, 클래스)

- **struct** : 연관된 여러 데이터를 하나의 구조로 묶는 문법이다. 클래스와 달리 내부 멤버의 기본 접근 제어 권한이 public(외부 접근 가능)이다.

```
struct Point {
    double x;
    double y;
};

Point pt;
pt.x = 10.5; // 외부에서 직접 데이터 멤버에 접근 가능
pt.y = 20.7;
```

- **class** : 객체를 생성하기 위한 속성(변수)과 행위(함수)를 묶은 설계도이다. 기본 접근 제어 권한이 private(외부 직접 접근 불가)로 설정된다.

- **접근 제어 지시자 (public / private)** : public은 외부에서 자유롭게 접근할 수 있게 열어두는 영역이고, private은 외부 직접 접근을 막아 데이터를 보호하는 영역이다.
- **캡슐화 (Getter / Setter)** : 내부 변수는 private에 숨기고, 이를 안전하게 읽고 쓰기 위한 public 함수(Getter, Setter)를 제공하는 설계 방식이다.

```
class Robot {
private:
    int speed; // private: 외부 직접 접근 불가

public:
    // constructor (생성자): 객체 생성 시 자동 실행되며 반환 타입을 쓰지 않는다
    Robot() : speed(0) {}

    // destructor (소멸자): 객체 소멸 시 자동 실행되며 클래스 이름 앞에 ~를 붙인다
    ~Robot() {}

    void setSpeed(int value) { // Setter: 값을 안전하게 변경
        if (value >= 0) speed = value;
    }

    int getSpeed() const { // Getter: 값을 외부로 읽어옴
        return speed;
    }
};
```

- **멤버 접근 연산자 (. / ->)** : 일반 객체의 멤버에 접근할 때는 점(.)을 쓰고, 포인터나 스마트 포인터 객체의 멤버에 접근할 때는 화살표(->)를 사용한다.

```
Robot robot;
robot.setSpeed(10); // 일반 객체는 점(.)으로 접근

Robot* ptr = &robot;
ptr->setSpeed(20); // 포인터 객체는 화살표(->)로 접근
```

모던 현대 C++ 문법

- **std::vector** : 데이터 개수에 따라 크기가 자유롭게 변하는 동적 배열 컨테이너이다.
- **push_back()** : std::vector 컨테이너의 맨 마지막 끝에 새로운 값을 추가하는 함수이다.
- **size()** : std::vector 컨테이너 내부에 저장된 총 원소의 개수를 확인하는 함수이다.
- **std::shared_ptr** : 동적 할당된 메모리를 수동으로 해제할 필요 없이 수명이 끝나면 자동으로 관리해 주는 안전한 스마트 포인터이다.
- **std::make_shared** : shared_ptr 객체를 안전하고 효율적으로 생성하는 함수이다.

- **auto** : 컴파일러가 우측 대입 값을 바탕으로 변수의 타입을 자동으로 추론해 주는 문법이다.
- **const** : 변수의 값이 중간에 임의로 변경되지 않도록 상수로 고정하여 수정을 방지하는 문법이다.
- **범위 기반 for문** : 인덱스 계산 없이 컨테이너 내부의 모든 값을 순차적으로 하나씩 꺼내어 반복하는 편리한 반복문이다.

```
// auto와 std::make_shared를 활용한 스마트 포인터 객체 생성
auto robot = std::make_shared<Robot>();
robot->setSpeed(15); // 화살표 연산자(->)를 통한 멤버 접근

// std::vector 활용 및 push_back(), size() 예시
std::vector<int> nums = {10, 20};
nums.push_back(30); // 맨 끝에 30 추가

std::cout << "원소 개수: " << nums.size(); // size()는 3을 반환

// 범위 기반 for문
for (int n : nums) {
    std::cout << n;
}
```

- **람다 (Lambda) 및 [this] 캡처** : 별도의 함수 이름을 정의하지 않고 코드 내부에서 즉석으로 선언하여 사용하는 익명 함수 문법이다. [this]를 명시하면 람다 식 내부에서도 현재 속한 클래스 객체의 멤버 변수나 함수에 접근할 수 있다.

```
// 람다 함수 형태의 예시 틀
[this]() {
    move(); // 현재 클래스 객체의 멤버 함수 호출 가능
};
```